Python for computational Problem Solving · Unit_01 · Unit 1: Introduction

DRAFT — UE26CS151A · Unit_01 — Unit 1: Introduction · Complete Unit Master

Notes-first study master · source-cited lecture explanations and compact page takeaways

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the edited lesson notes by topic. Evidence chips point to transcript intervals and slide pages. The complete source transcripts, page text, formula candidates, and image locators are preserved in sibling JSON datasets. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_01: Unit 1: Introduction
2. 2 · Introduction
3. 3 · Man Cabbage Goat Wolf Problem
4. 4 · Process of Computation Problem Solving - Analysis, Design, Implementation, Testing
5. 5 · Process of Computation Problem Solving - Analysis, Design, Implementation, Testing
6. 6 · Computer Hardware and software
7. 7 · Computer hardware
8. 8 · Program Structure
9. 9 · Program structure
10. 10 · Program structure
11. 11 · print function
12. 12 · Variable video 1
13. 13 · variable and id function
14. 14 · input function
15. 15 · input function
16. 16 · Arithmetic, membership and relational operators
17. 17 · Bitwise operators
18. 18 · logical operators and short circuit evaluation
19. 19 · operators and expressions
20. 20 · sequential, selection control
21. 21 · iterative control structure
22. 22 · String Operations
23. 23 · Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules
24. 24 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages

25. 25 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models
26. 26 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution
27. 27 · Source slide deck: IDLE
28. 28 · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type
29. 29 · Source slide deck: Operator Polymorphism
30. 30 · Source slide deck: Operator precedence and associativity
31. 31 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python
32. 32 · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists
33. 33 · Historical question-source pages · unit mapping candidates
34. Appendix A · Symbol glossary
35. Appendix B · Sources, coverage and gaps
36. Appendix C · Revision and retrieval practice

M1

Unit_01 · Unit 1: Introduction

1 UE26CS151A · Unit_01: Unit 1: Introduction

This unit master organizes source material for Python for computational Problem Solving, Unit_01, officially titled Unit 1: Introduction. It includes 21 video transcripts and 20 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review. EV0000001

SHORT-NOTE RECAP

Unit map · UE26CS151A · Unit_01: Unit 1: Introduction

Official catalogue entry: UE26CS151A, Unit_01 (Unit 1: Introduction). Bucket folder present: yes. Videos: 21; slide decks: 20.

EV0000001

2 Introduction

Study focus: Introduction. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000002 · EV0000003

SHORT-NOTE RECAP

Computational problem solving is a process, not just coding. Begin by understanding the goal and constraints, then represent the problem, choose a method, implement it in Python, and test the result against cases that matter.

Keep the problem statement separate from the program. A computer follows exact instructions; it does not infer missing requirements. Use examples to clarify input, output, and edge cases before writing code.

Python can be used interactively to explore an expression or in a saved `.py` file for a repeatable program. The course connects algorithmic reasoning to syntax, data, control flow, and testing; each later topic supplies a tool in that larger process.

EV0000002 · EV0000003 · EV0000004 · +59 source units in pack.json

3 Man Cabbage Goat Wolf Problem

Study focus: Man Cabbage Goat Wolf Problem. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000010 · EV0000011

SHORT-NOTE RECAP

Model the river puzzle as a state-space search. The farmer, wolf, goat, and cabbage begin on one bank. The boat carries the farmer plus at most one passenger. A state records which bank each item occupies.

A state is unsafe if the farmer leaves the wolf alone with the goat, or the goat alone with the cabbage. A legal move changes the farmer's bank and optionally moves one item with him; reject any move that creates an unsafe state.

Search the remaining legal states until the goal is reached, and keep a record of visited states to avoid cycling. The key computational idea is to make constraints explicit, generate only valid successors, and verify that the final state satisfies the goal.

EV0000010 · EV0000011 · EV0000012 · +9 source units in pack.json

4 Process of Computation Problem Solving - Analysis, Design, Implementation, Testing

Study focus: Process of Computation Problem Solving - Analysis, Design, Implementation, Testing. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000022 · EV0000023

SHORT-NOTE RECAP

Separate the stages of solving a computational problem. Analysis identifies the required result, inputs, constraints, and special cases. Design turns that understanding into an algorithm or data representation. Implementation expresses the design in a programming language.

Testing compares actual outputs with expected behavior on representative and boundary inputs. A syntax error means the program violates Python's grammar; a logic error means it runs but computes the wrong result. Passing one example is not proof of correctness, so use cases that challenge assumptions.

There may be several valid designs. Compare them against clarity, correctness, and resource requirements instead of treating the first working program as the only solution.

EV0000022 · EV0000023 · EV0000024 · +45 source units in pack.json

5 Process of Computation Problem Solving - Analysis, Design, Implementation, Testing

Study focus: Process of Computation Problem Solving - Analysis, Design, Implementation, Testing. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000034 · EV0000035

SHORT-NOTE RECAP

Separate the stages of solving a computational problem. Analysis identifies the required result, inputs, constraints, and special cases. Design turns that understanding into an algorithm or data representation. Implementation expresses the design in a programming language.

Testing compares actual outputs with expected behavior on representative and boundary inputs. A syntax error means the program violates Python's grammar; a logic error means it runs but computes the wrong result. Passing one example is not proof of correctness, so use cases that challenge assumptions.

There may be several valid designs. Compare them against clarity, correctness, and resource requirements instead of treating the first working program as the only solution.

EV0000034 · EV0000035 · EV0000036 · +17 source units in pack.json

6 Computer Hardware and software

Study focus: Computer Hardware and software. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000054 · EV0000055

SHORT-NOTE RECAP

Hardware is the physical machine; software is the instructions and data it uses. A computer accepts input, stores and processes data, and produces output. The processor executes instructions, memory holds active data and programs, and input/output devices connect the computer to its environment.

Operating-system software manages hardware resources and provides services to applications. Application software performs tasks for a user. A program is not the hardware itself: it is a representation of operations that the machine executes.

Distinguish primary memory, which the processor can access directly while programs run, from persistent secondary storage. The distinction helps explain where instructions and data reside at each stage of computation.

EV0000054 · EV0000055 · EV0000056 · +56 source units in pack.json

7 Computer hardware

Study focus: Computer hardware. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000074 · EV0000075

SHORT-NOTE RECAP

A digital computer processes encoded data through coordinated components. The input unit supplies data and instructions; memory stores them; the processor fetches and executes instructions; and output devices present results. The arithmetic and logic unit performs calculations and comparisons, while the control unit coordinates instruction execution.

Registers and main memory are fast working storage but are volatile: their contents normally disappear when power is removed. Secondary storage such as an SSD retains files. The processor does not directly understand a human problem; software encodes the steps and data in forms the machine can process.

When tracing a program, follow both the changing values in memory and the sequence of instructions executed by the processor.

EV0000074 · EV0000075 · EV0000076 · +9 source units in pack.json

8 Program Structure

Study focus: Program Structure. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000086 · EV0000087

SHORT-NOTE RECAP

A Python program is built from statements and expressions arranged with meaningful indentation. A statement performs an action, such as assigning a value or calling a function; an expression produces a value. Names identify variables and functions, while literals provide fixed values.

Indentation is part of Python's syntax: statements at the same indentation level belong to the same block, and a block follows a header ending in a colon. Comments explain intent to a human reader but are ignored by Python.

Keep a program readable by using descriptive names, consistent indentation, and small sections that each have a clear purpose. Read errors from the first reported line, then check nearby syntax and block boundaries.

EV0000086 · EV0000087 · EV0000088 · +19 source units in pack.json

9 Program structure

Study focus: Program structure. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000090 · EV0000091

SHORT-NOTE RECAP

A Python program is built from statements and expressions arranged with meaningful indentation. A statement performs an action, such as assigning a value or calling a function; an expression produces a value. Names identify variables and functions, while literals provide fixed values.

Indentation is part of Python's syntax: statements at the same indentation level belong to the same block, and a block follows a header ending in a colon. Comments explain intent to a human reader but are ignored by Python.

Keep a program readable by using descriptive names, consistent indentation, and small sections that each have a clear purpose. Read errors from the first reported line, then check nearby syntax and block boundaries.

EV0000090 · EV0000091 · EV0000092 · +17 source units in pack.json

10 Program structure

Study focus: Program structure. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000110 · EV0000111

SHORT-NOTE RECAP

A Python program is built from statements and expressions arranged with meaningful indentation. A statement performs an action, such as assigning a value or calling a function; an expression produces a value. Names identify variables and functions, while literals provide fixed values.

Indentation is part of Python's syntax: statements at the same indentation level belong to the same block, and a block follows a header ending in a colon. Comments explain intent to a human reader but are ignored by Python.

Keep a program readable by using descriptive names, consistent indentation, and small sections that each have a clear purpose. Read errors from the first reported line, then check nearby syntax and block boundaries.

EV0000110 · EV0000111 · EV0000112 · +17 source units in pack.json

11 print function

Study focus: print function. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000130 · EV0000131

SHORT-NOTE RECAP

`print()` writes a readable representation of values to standard output. It can display one value or several values separated by commas. The `sep` argument controls the text between values, and `end` controls what is written after them; by default, values are separated by a space and the call ends with a newline.

Strings need quotation marks in source code, but `print()` displays their contents without those delimiters. Printing is useful for user-facing output and for tracing a program while debugging.

Remember that printing a value is different from returning it from a function. Output is text sent to the terminal; a returned value can be used by later expressions.

EV0000130 · EV0000131 · EV0000132 · +34 source units in pack.json

12 Variable video 1

Study focus: Variable video 1. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000146 · EV0000147

SHORT-NOTE RECAP

A variable name refers to a value used by a program. Assignment evaluates the right-hand expression and binds the resulting value to the name on the left. Later assignment can bind that same name to another value.

Python is dynamically typed: the type belongs to the object, and a name can refer to objects of different types at different points in a program. Choose names that describe their role, and do not confuse the name with the storage location or with the value itself.

When tracing code, update a name's binding after each assignment. This simple state trace exposes errors caused by using an old or unexpected value.

EV0000146 · EV0000147 · EV0000148 · +9 source units in pack.json

13 variable and id function

Study focus: variable and id function. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000158 · EV0000159

SHORT-NOTE RECAP

Assignment binds a name to an object; it does not copy an object's identity into the name. The built-in `id(x)` returns an integer that is unique and constant for that object's lifetime. In CPython it often corresponds to a memory address, but Python code should not rely on that implementation detail.

Use `==` to compare values for equality. Use `is` only when object identity is what matters, most commonly to test a singleton such as `None`. Two separate objects can compare equal while having different identities.

For mutable objects, two names may refer to the same object; changing it through either name is visible through both. This is why identity and value equality must be kept distinct.

EV0000158 · EV0000159 · EV0000160 · +86 source units in pack.json

14 input function

Study focus: input function. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000166 · EV0000167

SHORT-NOTE RECAP

`input()` reads a line of text and returns it as a string. A prompt can be supplied to tell the user what to enter, but the returned value is still text. Convert it before arithmetic: use `int()` for an integer or `float()` for a decimal quantity, and handle invalid input when the program requires robustness.

Separate reading, conversion, validation, and computation. This makes it clear whether a failure came from unexpected text, an out-of-range value, or the algorithm itself.

Do not assume that typing digits makes the result numeric: `input()` returns a string until the program explicitly converts it.

EV0000166 · EV0000167 · EV0000168 · +28 source units in pack.json

15 input function

Study focus: input function. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000182 · EV0000183

SHORT-NOTE RECAP

`input()` reads a line of text and returns it as a string. A prompt can be supplied to tell the user what to enter, but the returned value is still text. Convert it before arithmetic: use `int()` for an integer or `float()` for a decimal quantity, and handle invalid input when the program requires robustness.

Separate reading, conversion, validation, and computation. This makes it clear whether a failure came from unexpected text, an out-of-range value, or the algorithm itself.

Do not assume that typing digits makes the result numeric: `input()` returns a string until the program explicitly converts it.

EV0000182 · EV0000183 · EV0000184 · +13 source units in pack.json

16 Arithmetic, membership and relational operators

Study focus: Arithmetic, membership and relational operators. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000198 · EV0000199

SHORT-NOTE RECAP

Operators combine operands to compute or test values. Arithmetic operators perform numeric operations; relational operators such as `<`, `==`, and `>=` compare values and produce a Boolean result; membership operators `in` and `not in` test whether a value occurs in a collection or string.

Keep the result type in mind: `5 > 2` is a Boolean, while `5 + 2` is numeric. Division with `/` produces a floating-point result in Python, even when both inputs are integers; `//` is floor division.

Use parentheses to make mixed expressions easy to read, and test boundary cases for comparisons.

EV0000198 · EV0000199 · EV0000200 · +90 source units in pack.json

17 Bitwise operators

Study focus: Bitwise operators. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000222 · EV0000223

SHORT-NOTE RECAP

Bitwise operators act on the binary representation of integers. AND keeps a bit only when both corresponding bits are 1; OR keeps it when either is 1; XOR keeps it when the bits differ; complement inverts the representation under Python's integer model. Left and right shifts move bit positions and correspond to multiplication or floor division by powers of two for nonnegative integers.

Bitwise operations are useful for masks, flags, and low-level encodings. They are different from Boolean `and`, `or`, and `not`, which work with truth values and short-circuit evaluation.

Write small binary examples before combining masks; the bit-by-bit view is the safest way to check a result.

EV0000222 · EV0000223 · EV0000224 · +13 source units in pack.json

18 logical operators and short circuit evaluation

Study focus: logical operators and short circuit evaluation. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000238 · EV0000239

SHORT-NOTE RECAP

Boolean logic combines conditions. `not` negates truth, `and` is true only when both operands are truthy, and `or` is true when at least one is truthy. Python evaluates `and` and `or` from left to right and may stop as soon as the result is determined.

With `A and B`, Python skips `B` if `A` is false. With `A or B`, it skips `B` if `A` is true. This short-circuit rule supports safe guards such as checking that a value is not `None` before using it.

These operators return one of their operands, not necessarily a Boolean. Use explicit comparisons when the code's intent would otherwise be unclear.

EV0000238 · EV0000239 · EV0000240 · +5 source units in pack.json

19 operators and expressions

Study focus: operators and expressions. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000246 · EV0000247

Key terms and definitions

Expression. A syntactically valid combination of values, names, and operators that Python evaluates to a value. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

Operand. A value or expression on which an operator acts. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

Precedence. The priority that determines which operator binds first in an expression. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

Associativity. The grouping direction used when operators of equal precedence occur together. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

Worked examples

W000001 · Apply precedence and associativity

GIVEN	Evaluate `2 + 3 * 4`, `10 - 3 - 2`, and `2 ** 3 ** 2`.
METHOD	Apply Python operator precedence and associativity.
STEP 1	Multiplication precedes addition: `2 + (3 * 4) = 14`.
STEP 2	Subtraction is left associative: `(10 - 3) - 2 = 5`.
STEP 3	Exponentiation is right associative: `2 ** (3 ** 2) = 512`.
STEP 4	Parentheses make each grouping explicit and are useful even when the default rule is known.
ANSWER	14, 5, and 512, respectively.
UNITS	Dimensionless integers.
CHECK	The results follow Python's operator precedence table and documented associativity rules; no source slide value has been assumed.

EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

SHORT-NOTE RECAP

Expressions evaluate to values. A literal such as `123`, `3.5`, or a string is already an expression. A variable name is also an expression because evaluating the name retrieves its current value. Operators combine operands to build larger expressions.

Operators and operands. A unary operator acts on one operand, a binary operator on two, and Python's conditional expression has three parts: `value_if_true if condition else value_if_false`. Parentheses explicitly group a subexpression.

Precedence chooses which operation binds first. In `2 + 3 * 4`, multiplication has higher precedence, so Python evaluates `3 * 4` first and returns `14`. Parentheses can override the default grouping; use them when the intended order should be obvious to a reader.

Associativity breaks ties. Operators at the same precedence level follow their associativity rule. Subtraction groups left to right, so `10 - 3 - 2` means `(10 - 3) - 2 = 5`. Exponentiation groups right to left, so `2 ** 3 ** 2` means `2 ** (3 ** 2) = 512`. Do not assume every operator groups in the same direction.

Where the result appears. The interactive interpreter echoes the value of a bare expression. A saved `.py` program needs `print(expression)` when the result should appear in the terminal.

EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

20 sequential, selection control

Study focus: sequential, selection control. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000286 · EV0000287

SHORT-NOTE RECAP

Sequential control runs statements in order; selection chooses a branch. Python executes a block from top to bottom unless a control statement changes the path. An `if`/`elif`/`else` chain evaluates conditions in order and executes the first matching branch.

Write conditions that cover the intended cases without overlap where possible. Put the most specific conditions before broader ones, and include an `else` branch when every remaining case needs a defined response.

Indentation marks each branch's body. Trace a few inputs by recording which condition is tested and which statements run; this is more reliable than guessing from the visual layout.

EV0000286 · EV0000287 · EV0000288 · +73 source units in pack.json

21 iterative control structure

Study focus: iterative control structure. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000302 · EV0000303

SHORT-NOTE RECAP

Iteration repeats a block while progressing toward a stopping condition. A `for` loop visits items from an iterable; a `while` loop repeats as long as its condition is true. For each loop, identify initialization, the repeated work, the state update, and the exit condition.

A `for` loop is often the clearest choice when processing a known collection or range. A `while` loop is useful when the number of repetitions depends on a changing condition. Make sure every path through a while-loop body moves toward termination.

`break` exits the nearest loop and `continue` skips to its next iteration. Use them sparingly so the loop's flow remains easy to trace.

EV0000302 · EV0000303 · EV0000304 · +17 source units in pack.json

22 String Operations

Study focus: String Operations. The notes explain the key terms, distinctions, and practical method used in this lesson. Read the cited transcript intervals for source context, then use the examples to check your understanding. EV0000322 · EV0000323

SHORT-NOTE RECAP

A Python string is an immutable sequence of Unicode characters. Indexing selects one character, slicing selects a half-open range, and concatenation combines strings. A slice `s[a:b]` includes index `a` and stops before index `b`.

Common methods return new strings: `lower()` changes letter case, `strip()` removes edge whitespace, and `split()` separates text into a list. Because strings are immutable, a method does not modify the original string in place.

Check index boundaries and empty strings when processing input. For readable output that mixes text and values, use formatted string literals rather than manual concatenation.

EV0000322 · EV0000323 · EV0000324 · +95 source units in pack.json

23 Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules

Study focus: Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000330 · EV0000331

SHORT-NOTE RECAP

Study notes · Source slide deck: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Prof. Sowmya Shree P, Prof. Sindhu R Pai

Slide 2 · PROBLEM SOLVING

- Unit - 3: Modules – Import Mechanisms
- Prof. Sowmya Shree P, Prof. Sindhu R Pai
- Department of Computer Science and Engineering

Slide 3 · Modules – Import mechanisms

- In Python, a module is a file that contains code.
- It can include functions, classes, variables or any runnable code.
- We can use modules to separate codes in separate files as per their

Slide 4 · Modules – Import mechanisms

- Modularity – Organizing the code into modules logically
- Separate scopes – separate namespace is defined by the module
- Grouping - Python modules help us to organize and group the content

Slide 5 · Modules – Import mechanisms

- If we include everything in a single file, the program may become large.
- To reduce size of the code, we can group together some similar
- That collection is nothing but modules in python.

Slide 6 · Modules – Import mechanisms

- Modules in Python can be grouped together in packages.
- namespace to the modules so that they don't conflict with modules
- The import statement can also be used to import modules from a

Slide 7 · Modules – Import mechanisms

- Modules – contain several functions, variables, classes etc.
- folder that contains various modules as files.
- Library - a collection of packages and modules used to access built-in

Slide 8 · Modules – Import mechanisms

- Python's standard library comes bundled with a large number of
- They are called built-in modules.
- Any file with .py extension and containing Python code is basically

Slide 9 · Modules – Import mechanisms

- os provides a unified interface to a number of operating system
- string contains a number of functions for string processing
- cmath a number of mathematical operations for complex numbers

Slide 10 · Modules – Import mechanisms

- operator Functions on the standard operators
- pickle Convert Python objects to streams of bytes and back
- sqlite3 A DB-API 2.0 implementation using SQLite 3.x

Slide 11 · Modules – Import mechanisms

- Any file with .py extension and containing Python code is a module.
- Any object from a module can be made available to interpreter or
- another Python script by using import statement.

Slide 12 · Modules – Import mechanisms

- Crete a file with .py extension
- Example: Creating a module(module1.py)

- print("Welcome to Module-1")

Slide 13 · Modules – Import mechanisms

- import the functions, and classes defined in a module to another
- When the interpreter encounters an import statement, it imports the
- module if the module is present. Otherwise, ModuleNotFoundError is

Slide 14 · Modules – Import mechanisms

- Example: importing the module1(refer previous slide)
- import module1 #All functions and variables of module1 are available
- print("value is", module1.a)
- print("value is", module1.a)

Slide 15 · Modules – Import mechanisms

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 16 · Modules – Import mechanisms

- Import Specific Attributes from a module
- Syntax – from module_name import specific_attributes
- return (x/y) #Assuming 'y' is never zero

Slide 17 · Modules – Import mechanisms

- from module2 import add,multiply
- #importing only add and multiply functions from module2
- print("Sum=",add(10,20))

Slide 18 · Modules – Import mechanisms

- Import all Names from a Module
- symbol with the import statement is used to import all the names from a
- Syntax - from module_name import

Slide 19 · Modules – Import mechanisms

- Import all Names from a Module
- #importing all the functions from module2
- Note: If we know exactly which attribute to import from the module, it is not recommended to use import

Slide 20 · Modules – Import mechanisms

- We can rename the module while importing it.
- Syntax – import module_name as alias_name
- import math as mt #Renaming math module as 'mt'

Slide 21 · Modules – Import mechanisms

- print("Illustration of Renaming a Module")
- print("Acceleration due to gravity on earth=",m5.GRAVITY,"m/s\u00b2")
- Acceleration due to gravity on earth= 9.8 m/s²

Slide 22 · Modules – Import mechanisms

- If not built-in module, Search for the Module in the current directory
- If not found in current directory, Python then searches each directory
- If that also fails python checks the sys.path (A built-in variable within

Slide 23 · Modules – Import mechanisms

- ['', 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\idlelib',
- 'C:\\Users\\SOWMYA SHREE P\\AppData\\Local\\Programs\\Python\\Python311\\python311.zip',
- This is a list of directories that the interpreter will search for the required

Slide 24 · Modules – Import mechanisms

- a built-in function of sys module that can be used with path variable to add a
- a built-in function of sys module that can be used to insert a path at a specific
- 0 indicates that the path should be inserted at the beginning of sys.path .

Slide 25 · Modules – Import mechanisms

- In Python, each object(Class, Fucntion, variable,..) can be documented
- Docstrings can be accessed using the __doc__ attribute.
- print("Using __doc__:") Help on function add in module __main__:

Slide 26 · Modules – Import mechanisms

- It is a special variable in Python.
- If the source file is executed as the main program, the interpreter sets the
- If this file is being imported from another module, __name__ will be set to

Slide 27 · Modules – Import mechanisms

- print (“file1 __name__ = %s" %__name__)
- print (“file1 is executed directly")
- After executing file1.pyfile1 __name__ = __main__

Slide 28 · Modules – Import mechanisms

- print ("file2 __name__ = %s" %__name__)
- print ("file2 is executed directly")
- file1 is imported After executing file2.py

Slide 29 · Modules – Import mechanisms

- Python Module is a python script file that can contain variables,
- Python modules help us in organizing our code and then referencing
- tasks. These sub-tasks are Modules.

Slide 30 · Department of Computer Science and Engineering

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

EV0000330 · EV0000331 · EV0000332 · +87 source units in pack.json

24 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages

Study focus: Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0000630 · EV0000631

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Introduction to Programming Languages

- ☒Translates human logic to a form computers can understand
- ☒Instructions communicated effectively. Thanks to syntax (set of rules) provided by PLs
- ☒Why: Computers only understand binary (0s and 1s). PLs act as a bridge between

Slide 3 · Introduction to Programming Languages

- Why so many programming languages?
- different applications (e.g., web development, system programming, data analysis)
- Efficiency and Flexibility: Some languages are better suited for certain tasks. For example,

Slide 4 · Introduction to Programming Languages

- Measure of popularity of programming languages, updated once a month
- Uses online search engine results to track language trends over time

- NotNot aboutabout whichwhich programmingprogramming languagelanguage isis “thethe best”best

Slide 5 · Introduction to Programming Languages

- The TIOBE Index (as of September 2024)

Slide 6 · Introduction to Programming Languages

- Low Level: Directly understood by machine. Much harder to code with, but has
- Middle Level: Provides some basic data structures and definitions while still
- High Level: Focuses on ease of use and provides many programming structures

Slide 7 · Introduction to Python

- ☒First published in Feb 1991 by Guido van Rossum
- ☒As of Sept 2024, Python Package Index contains more than 565,000 packages
- ☒Versatile uses: GUIs, test frameworks, automation and web scraping, scientific

Slide 8 · Applications of Python

- Data Science: Libraries like NumPy, Pandas, Matplotlib etc. are used for predictive
- AI & Machine Learning: Python is used to develop neural networks and NLP systems
- Drug Discovery: Python is used in molecular modelling and simulations using libraries

Slide 9 · Applications of Python

- IoT and Embedded Systems: Integrations with platforms like Raspberry Pi and Arduino
- CircuitCircuit DesignDesign andand Simulation:Simulation: ToolsTools likelike PySpicePySpice, SKiDLSKiDL areare usedused forfor circuitcircuit simulationsimulation
- Computational Fluid Dynamics (CFD): Libraries such as OpenFOAM are used for

Slide 10 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shyalaja S S, Director, CCBD and CDSAML, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu
- Ack: Teaching Assistant – Advaith Sanil Kumar

EV0000630 · EV0000631 · EV0000632 · +27 source units in pack.json

25 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models

Study focus: Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0000660 · EV0000661

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Syntax, Semantics and Translation Models
- Department of Computer Science and Engineering

Slide 2 · the alphabet,• English, for example, includes the letters of punctuation and properly

- The syntax of a language is a set of characters and the acceptable sequences
- The following is a syntactically correct sentence in English,
- The following, however, is not syntactically correct,

Slide 3 · Consider the following sentence: I will read yesterday

- This sentence is syntactically correct, but has no meaning. It is semantically incorrect.
- The semantics of a language is themeaning associated with each syntactically
- Note: Every language has its own syntax and semantics.

Slide 4 · ☒A programming language provides the necessary constructs to instruct the computer to

- do the tasks for us.
- ☒Translation models describe the mathematical relationship between two or more
- ☒Models of translational equivalence - Whether expressions in different languages

Slide 5 · Language Translators

- CompilerCompiler convertsconverts entireentire highhigh-levellevel languagelanguage programsprograms totoo machinemachine languagelanguage atat onceonce.
- Interpreter converts high-level language to machine language line by line and execute
- instructions in place of CPU

Slide 6 · Slide page 6

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 7 · Slide page 7

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 8 · Introduction to Computer Science Using Python – Dierbach

- Copyrighth 2013 Johnh Wileyl andd Sons
- Department of Computer Science and Engineering
- Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

EV0000660 · EV0000661 · EV0000662 · +21 source units in pack.json

26 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution

Study focus: Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0000684 · EV0000685

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Requirements and Installation

- Windows, macOS and Linux Visit the Python Official Website and
- Windows: Run the installer and and double click the downloaded Python installer file. Tick the checkbox. Click
- Mac: Double-click the .pkg file to start the installation. Click “Continue” and follow the prompts.

Slide 3 · Different ways of Python code execution

- Batch mode/script/ p mode/File/ mode
- Using IDLE - Integrated Development and Learning Environment

Slide 4 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shyalaja S S, Director, CCBD & CDSAML, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000684 · EV0000685 · EV0000686 · +9 source units in pack.json

27 Source slide deck: IDLE

Study focus: Source slide deck: IDLE. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000714 · EV0000715

SHORT-NOTE RECAP

Study notes · Source slide deck: IDLE

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Solving- UE26CS151A

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · IDLE – Python Development Environment

- IDLE stands for Integrated Development and Learning
- It is a simple Python development environment that comes with
- It helps users write, run, test, and debug Python programs.

Slide 3 · IDLE – Python Development Environment

- Python Shell for interactive execution
- Code editor for writing programs

Slide 4 · IDLE – Python Development Environment

- ◦ Go to the official Python Website and click on the download link
- ◦ that says "Add python.exe to PATH" (if available).
- ◦ Click on "Install Now". This default setting installs Python along

Slide 5 · IDLE – Python Development Environment

- Wait for the progress bar to finish until a "Setup was successful”
- 2. Type IDLE in the search bar.
- 3. Click on IDLE (Python 3.x) to launch the application

Slide 6 · IDLE – Python Development Environment

- When IDLE opens, the Python Shell is displayed.
- The Shell provides the >>> prompt for entering Python statements.

Slide 7 · IDLE – Python Development Environment

- Debugger and other development tools

Slide 8 · IDLE – Python Development Environment

- The Python Shell executes Python statements interactively.

Slide 9 · IDLE – Python Development Environment

- Executes one statement at a time.
- Useful for learning, calculations, testing small code
- fragments, and experimenting with Python.

Slide 10 · IDLE – Python Development Environment

- Used to write complete Python programs.
- Programs are normally saved with the .py extension.
- A complete program can be edited, saved, and

Slide 11 · IDLE – Python Development Environment

- Write the Python program in the Editor window.
- Save using a meaningful .py filename.

Slide 12 · IDLE – Python Development Environment

- IDLE executes the current module.
- Output is displayed in the Python Shell.

Slide 13 · IDLE – Python Development Environment

- Interactive Mode vs Script Mode
- Executes one line at a time Executes complete program
- Immediate output Saved as .py file

Slide 14 · IDLE – Python Development Environment

- Syntax highlighting is a feature in IDLE that displays different parts of Python
- code in different colors, making the code easier to read and understand.
- Helps beginners understand Python code more easily.

Slide 15 · IDLE – Python Development Environment

- Python uses indentation to define blocks.
- IDLE assists with automatic indentation.
- Indentation is part of Python syntax.

Slide 16 · IDLE – Python Development Environment

- Script Mode: Saving Python Programs
- Python source files use the .py extension.
- Use File → Save or Ctrl + S.

Slide 17 · IDLE – Python Development Environment

- Script Mode: Opening an Existing Program
- The file opens in the IDLE Editor Window.
- You can edit, save, and run it using F5.

Slide 18 · IDLE – Python Development Environment

- IDLE displays error messages when problems occur.
- SyntaxError Invalid Python syntax print(“Hello"
- IndentationErr Incorrect indentation in Missing spaces after if

Slide 19 · IDLE – Python Development Environment

- Built-in debugger included with IDLE.
- Executes programs step by step.
- Allows you to inspect variable values during execution.

Slide 20 · IDLE – Python Development Environment

- A breakpoint is a marker that temporarily pauses program execution at a
- Observe the flow of the program step by step.
- Saves time when troubleshooting complex code.

Slide 21 · IDLE – Python Development Environment

- New File Ctrl + N ⌘+ N
- Open File Ctrl + O ⌘+ O
- Save File Ctrl + S ⌘+ S

Slide 22 · IDLE – Python Development Environment

- Example: Area of a Circle
- radius = float(input("Enter radius: "))
- Example output: Enter radius: 5 → Area =

Slide 23 · IDLE – Python Development Environment

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 24 · IDLE – Python Development Environment

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 25 · IDLE – Python Development Environment

- Less suitable for very large projects
- Advanced users may prefer feature-rich IDEs such as VS Code or

Slide 26 · IDLE – Python Development Environment

- IDLE stands for Integrated Development and Learning

- The Editor Window is used to write and run Python scripts.
- IDLE includes basic debugging tools to help find and fix

Slide 27 · Department of CSE (AIML)

- Dr Manjula K- manjulak@pes.edu
- Ack: Teaching Assistant - Chiranth J(CSE AIML)

EV0000714 · EV0000715 · EV0000716 · +78 source units in pack.json

28 Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type

Study focus: Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000816 · EV0000817

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · An identifier is a sequence of one or more characters used to provide a name for a given

- program element. Examples: name , srn_number, ph_no, marks1, marks2
- identifier names. The variables that begin with underscore has a special meaning in object oriented
- programming. So we do not prefer to use _ as the first character.

Slide 3 · Valid and invalid Identifiers

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 4 · Keywords are reserved words that have a predefined meaning.

- To know the keywords, type help() in the python prompt and in the help prompt,

Slide 5 · A variable is a name (identifier) that is associated with a value and it is always reference

- A variable can be assigned different values during a program's execution—hence, the name
- Whenever variable num appears in a calculation, it is the current value of num
- If variable num is assigned a new value, then the same expression will produce a different

Slide 6 · Diagrammatic Representation

- Variables are assigned values by use of the assignment operator , =
- Note: The right side of an assignment is evaluated first, then the result is assigned to the

Slide 7 · Diagrammatic Representation

- Variables may also be assigned to the value of another variable.
- Note: Variables num and k are both associated with the same literal value 10 in memory. You
- Note: If no other variable references the memory location of the original value, the memory

Slide 8 · Diagrammatic Representation

- If the value of num changed, would variable k change along with it?
- Here variables refer to integer values, and integer values are immutable.
- Thus, both will continue to refer to the same value until one (or both) of them

Slide 9 · id function – id()

- This built-in function that returns the identity of an >>> num=10
- CommonlyCommonly usedused toto checkcheck ifif twotwo variablesvariables oror objectsobjects >>> num is k
- Falsereturn True if the two objects are the same else it

Slide 10 · Data Types

- Datatype refers to the type of value a variable refers to.
- Operations that can be performed on it.
- Range of values allowed in it

Slide 11 · Type function – type()

- A built-in function, that returns the type of the object
- Type of a variable depends on the value assigned to it

Slide 12 · Introduction to Computer Science Using Python – Dierbach

- Copyrighth 2013 Johnh Wileyl andd Sons
- Department of Computer Science and Engineering
- Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

EV0000816 · EV0000817 · EV0000818 · +33 source units in pack.json

29 Source slide deck: Operator Polymorphism

Study focus: Source slide deck: Operator Polymorphism. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000936 · EV0000937

SHORT-NOTE RECAP

Study notes · Source slide deck: Operator Polymorphism

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · PROBLEM SOLVING

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 3 · PYTHONPYTHON FORFOR COMPUTATIONALCOMPUTATIONAL PROBLEMPROBLEM SOLVINGSOLVING

- Operator polymorphism is a type of polymorphism where an operator behaves
- differently based on the type of the operands.

Slide 4 · Operator polymorphism

- + acts as addition operator if the operands are numbers.
- + acts as concatenation operator if the operand are strings.
- Note: Boolean literal True represents value 1 and False represents value 0

Slide 5 · Operator polymorphism

- Works as multiplication operator if the operands are numbers.
- Works as repetition operator if one operand is a string and other operand a

Slide 6 · gayathrirs@pes.edu

- Department of Computer Science and Engineering

EV0000936 · EV0000937 · EV0000938 · +15 source units in pack.json

30 Source slide deck: Operator precedence and associativity

Study focus: Source slide deck: Operator precedence and associativity. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000954 · EV0000955

SHORT-NOTE RECAP

Study notes · Source slide deck: Operator precedence and associativity

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · Operator Precedence

- Each programming language has its own rules for the order that operators are applied,
- There are two possible ways in which it can be evaluated
- 4 + 3 * 5 → 4 + 15 → 19 4 + 3 * 5 → 7 * 5 → 35

Slide 3 · Operator Precedence

- Operator precedence guarantees a consistent interpretation of expressions
- has higher precedence than+. Therefore the expression will be evaluated as follows.
- It is good programming practice to use parentheses even when not needed

Slide 4 · Operator Associativity

- Operator associativity defines the order that it and other operators with the
- the associativity of exponentiation operator is right to left.
- Therefore, 2**3**2 will be evaluated as follows

Slide 5 · Operator Precedence and Associativity

- The following table summarizes the operator precedence in Python, from highest
- precedence and have a left-to-right chaining feature.
- To see the complete documentation of operator precedence execute the below statement

Slide 6 · Operator Precedence

- "+x", "-x", "~x" Positive, negative, bitwise NOT
- "*", "/", "//", "%" Multiplication, division, floor division, remainder
- "in", "not in", "is", "is not", "<", Comparisons, including membership tests and identity tests

Slide 7 · Prof. Gayathri R S

- Department of Computer Science and Engineering

EV0000954 · EV0000955 · EV0000956 · +18 source units in pack.json

31 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python

Study focus: Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator.

EV0001035 · EV0001036

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · A way of organizing and storing data so that they can be accessed and worked with

- BecomesBecomes moremore significantsignificant withwith increaseincrease inin thethe volumevolume ofof datadata toto bebe storedstored andand
- Organization of books in library
- File system on a computer

Slide 3 · Data Structures

- Broad Classification of Data Structures
- Can be used to develop any collections
- There is a particular way in which an element can be accessed

Slide 4 · Data Structures

- Data stored in contiguous manner
- Can store homogeneous or heterogeneous data
- Non-Sequence Type Data Structures

Slide 5 · Data Structures

- Common Operations on Sequence Type

Slide 6 · Data Structures

- Common Operations on Non-Sequence Type
- Few of the relational operators

Slide 7 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0001035 · EV0001036 · EV0001037 · +18 source units in pack.json

32 Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists

Study focus: Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0001146 · EV0001147

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · A list is a collection that allows us to put many things / values under a single

- Values in the list are called elements / items.
- List is an ordered sequence of items.
- List is a linear data structure where elements have linear ordering.

Slide 3 · List Characteristics

- Elements in the list can be heterogeneous (different datatypes) or homogeneous
- Elements are accessed using indexing operation (also called subscript notation).
- List index always starts >>> print (numbers[1])

Slide 4 · List Characteristics

- List is iterable - is eager and not lazy.
- AssignmentAssignment ofof oneone listlist totoanotheranother causescauses bothboth totoreferrefer totothethe samesame listlist.
- List can be sliced. This creates a new (sub)list.

Slide 5 · List Characteristics

- Lists are mutable, as list can grow or shrink .
- o We can change an element of a list.
- >>> numbers[0]=10 # index operation is used.

Slide 6 · List Characteristics

- List is iterable - is eager and not lazy.
- Ex: (ii) number=[10,20,30,40,50] Ex: (i) numbers=[55,88,45,12]
- List can be nested. We can have list of lists.

Slide 7 · List Characteristics

- Assignment of one list to another causes both to refer to the same list.
- Ex: >>> list1=[12,44,55,89,11,24]
- Note: In Python, the id() function is a built-in function that returns the unique identifier of an object.

Slide 8 · List Characteristics

- List can be sliced. This creates a new (sub)list.
- Given lst1 = [12,44,55,89,11,24]
- >>> llst22 = llst1[::]1[ ] ## creates a copy off llst1.1 NNot same as llst22 = llst11

Slide 9 · Ex: >>> lst = [9, 41, 12, 3, 74, 15] >>> lst1=[10,20,[30,40,50]]

- >>> lst[1:3] >>> lst1[2][1]
- [9, 41, 12, 3, 74, 15]

Slide 10 · Creation of Lists

- List items are surrounded by square brackets and the elements in the list are
- A list element can be any Python object - even another list.
- politicians=['modi', 'jayalalitha', 'yediyurappa','devegowda',['parikar', 'swaraj',

Slide 11 · Built In Functions

- There are a number of functions built into Python that take lists
- >>> numsnums = [3,[3, 41,41, 12,12, 9,9, 74,74, 15]15]

Slide 12 · List Operations

- We can create a new list by adding two existing lists together.
- >>> list1 = [10,20,30,40,50]
- [10,20,30,40,50, 100,200,300,400,500]

Slide 13 · List Operations

- Allows for the multiplying of the list n times
- >>> list1 = [10,20,30,40,50]
- [10,20,30,40,50,10,20,30,40,50]

Slide 14 · List Operations

- Used to arrange the elements of a list.
- EEx: >>>>>> lilist1t1 = [10[10, 11, -2,2 22, 9]9]
- [-2, 1, 2, 9, 10]

Slide 15 · List Operations

- allows to add element at the end of list.

- >>> list1 = [10,20,30,40,50]
- [10,20,30,40,50,22]

Slide 16 · List Operations

- Allows to add an element at particular position in the list.
- >>> list1 = [10,20,30,40,50]
- [10,20,30,55,40,50]

Slide 17 · List Operations

- adds the specified list elements (or any iterable) to the
- >>>list1.extend([11,22,33,44,55])
- [10,20,30,40,50,11,22,33,44,55]

Slide 18 · pop() & remove()

- allows to remove element from a list by using pop() or remove() functions.
- One uses index value (pop),another uses value(remove) as reference to remove
- >>>list1.pop(2) # using pop() >>> list1.remove(40) #using remove()

Slide 19 · List Operations

- returns number of occurrences of value.
- >>> list1 = [10,20,30,40,50]

Slide 20 · List Operations

- return first index of a value. Raises ValueError if
- the value is not present.
- >>> list1 = [10,20,30,40,50]

Slide 21 · List Operators

- in returns True if a particular item exists in the list. otherwise False
- not in operator returns True if the element is not present, otherwise False
- Ex: >>> list1 = [10,2.2,(22,33,43),('python')] # heterogeneous list.

Slide 22 · We may at times need to compare data items in the two lists to perform certain operations

- >>> list1 = [10,2.2,(22,33,43]

Slide 23 · List Traversal

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 24 · 1. List using For Loop:

- ☒ The for loop in Python is used to iterate over a sequence (list, tuple, string) or other

- ☒ Loop continues until we reach the last item in the sequence
- ☒ The body of for loop is separated from the rest of the code using indentation.

Slide 25 · Slide page 25

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 26 · 2. List using while loop:

- The while loop in Python is used to iterate over a block of code as long as the test expression
- When the condition is tested and the result is false, the loop body will be skipped and the
- first statement after the while loop will be executed.

Slide 27 · Example: Sum of elements in list

- a=[1,2,3,4,5]

Slide 28 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CCBD &CDSAML, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu
- Ack: Teaching Assistant – Advaith Sanil Kumar

EV0001146 · EV0001147 · EV0001148 · +81 source units in pack.json

33 Historical question-source pages · unit mapping candidates

Study focus: Historical question-source pages · unit mapping candidates. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0001230 · EV0001231

SHORT-NOTE RECAP

Study notes · Historical question-source pages · unit mapping candidates

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Archive page 1 from Dec__1aef5kbQ-b.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 1/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - DEC 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Classify & group the following devices into Input/ Output/ Processing/ Storage/ Communication devices. Central Processing Unit, Modem, USB Stick, Microphone, Blue-Ray device, Speakers, Graphics Processing Unit, Touch screen, Projectors,

Blue-tooth. 1.b. Evaluate the following expressions: i) a='rama'>'ravana'; print(a) ii) print("1" * 2, 2 * "1") iii) a = 20; b = 5; print(a&b) iv) 5(2 + 3 * 4) v) import math; print(math.sqrt(-25)) 1.c. Write a Python program that prompts the user to enter positive integer values, and continues to prompt the user until a value of -1 is entered, adding up all the value entered and displaying the result. Input = Microphone, Microphone, Output = Blue-ray, Projector, Speakers Processing = CPU, GPU, Storage = USB stick Comms = Modem, Blue-tooth False 11 11 4 TypeError: 'int' object is not callable or 70 ValueError: math domain error

Archive page 2 from Dec__1aef5kbQ-b.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 2/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Explain the following: Algorithm Compiler Interpreter Semantic error Application software 2.a. Mention the properties of the list data structure. 2.b. Given s="python programming". What is the output of the following statements i) print (s[:]) ii) print (len(s)) iii) print (s[2:8]) iv) s[0]='T' print (s) v) k=s[0:6]+"not"+s[7:] print (k) 1 A sequence of unambiguous instructions for solving a problem 2 Compiler converts entire high-level language programs to machine language at once. 3 Interpreter converts high-level language to machine language line by line and execute instructions in place of CPU 4 a type of error that occurs when a program's meaning is incorrect, even though it runs successfully basically Logic error 5 Application software is used by user to perform a specific task. Mutable Indexing & slicing works ordered Iterable uses [ ] 1 python programming 2 18 3 thon p 4 they forgot ; ig. string immutable so TypeError 5 they forgot ; ig. pythonnotprogramming

Archive page 3 from Dec__1aef5kbQ-b.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 3/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Given a dictionary of student marks, create another dictionary that has the total marks of each student. Given, students= { 'Raja':[90,89,85,90], 'Ram':[89,87,80,91], 'Mohan':[91,88,88,92] } Sample output: Total_marks= { 'Raja': 354, 'Ram': 347, 'Mohan':359 } 2.d. Explain the following file functions. read() read(5) readlines() readline() write() 3.a. Write the function cumulative_sum which takes a list of numbers and returns a list of numbers obtained by adding successive elements. If the input list has n elements, the resulting list also have n elements. The first element is obtained by adding 0 to the first element of the input list. l1 = [10, 15, 30, 50] l2 = cumulative_sum(l1) print(l2) # should print [10, 25, 55, 105] 1 reads the entire file and returns a string 2 reads 5 characters/bytes and returns a string 3 reads the entire file and returns a list with each element: a string of each line 4 reads a single line each time and return a string (each time called moves to next line) 5 write data to a file

Archive page 1 from July__17cJZWr8dZ.pdf; unit match candidate score 0.8333; source text/OCR may be partial. (4.0 Marks) (7.0 Marks) (4.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JULY - 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Explain the stages in the computational problem-solving process. 1.b. Answer the following: i) _____ is a built-in function that is used to determine the type of a Python object. ii) print(len(["ChatGPT", "Bard", 10, 2, 7.3, 20,77.0])). iii) print("My", "last", "Exam ", sep="*") iv) print(5 + (2 ** 5) // 10) v) The _____ operator is used for integer division. vi) Which function returns a random integer in range [a, b], including both end points in random module. (a and b are the start and end values for the range). vii) 7+5i. (State if the decimal literal is valid or invalid in python.) 1.c.

Given the start and end numbers, write a python program to display all even numbers between the range. Sample Output: Enter the start of range: 6 Enter the end of range: 12 6 8 10 12 7 type() 8 My*last*Exam // import random random.randint Invalid

Archive page 2 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.d. A student's marks for five distinct subjects are taken using user input. Calculate and display the mark percentage. 2.a. Write a python program to read a string of 4 characters in lowercase from the user. Print every character of a string along with its uppercase letter and its ASCII/Unicode. Sample Output: Enter the string of 4 characters in lowercase: abcd a A 65 b B 66 c C 67 d D 68 2.b. Write a python program that takes the file name from the user and reads the contents(lines) of the file in the reverse order (from bottom). File1.txt contents: The laughter filled the room, infectious and joyous. It echoed and echoed, spreading happiness all around. It danced on the airwaves, filling hearts with warmth and light. Sample output: Enter file name: File1.txt It danced on the airwaves, filling hearts with warmth and light. It echoed and echoed, spreading happiness all around. The laughter filled the room, infectious and joyous.

Archive page 3 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (6.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Find the output of the following code snippets. i) t1 = (10,20,30,40,50) t2 = t1 + (60) t1 ii) t1=(100,200) t1*2 iii) d1=dict([(1,"Python"),(2,"Exam"),(3,"Over")]) print(d1,type(d1)) 2.d. Give any 5 built in functions of list in a tabular form with its description and code snippets 3.a. Write a function that accepts a string and calculate the number of upper-case letters and lower case letters in it. TypeError as (60) is not tuple its jut int. correct should be (60,) nothing gets printed but like it gives (100,200,100,200) pop,remove,clear,count,index

Archive page 6 from July__17cJZWr8dZ.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (3.0 Marks) 5.b. List the keywords in python with respect to Exception handling. 5.c. Find the output for the following python code. class Car: make = "TATA" def __init__(self,name,mileage): self.name = name self.mil = mileage c1 = Car("Harrier",18) c2 = Car("Nexon",25) print(c1.name,c1.mil,c1.make) c2.name = "i20" c2.make = "Hyundai" print(c2.name,c2.mil,c2.make) 5.d. Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list (if any). Sample input: L=[10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80] try, finally, else, except, raise Harrier 18 TATA i20 25 Hyundai

Archive page 1 from May__1SdbvafL_8.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - May 2024 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of each statement when executed separately. i) a = 40; print("a") ii) print(c == 0) iii) a = 4; b = a; print(id(a) == id(b)) iv) print(2 * 3 * 2) v) print(False and True) 1.b. 1) Provide the binary equivalent of a given decimal number - 53 2) Define

Translator. Name any two types of translators. 1.c. Write a program to print the following pattern. A B C D B C D C D D 1 a 2 NameError c not defined 3 True 4 12 5 False 110101 Translator translates between two or more languages compilers, interpreters

Archive page 2 from May__1SdbvafL_8.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 1.d. Write a neat diagram to explain the phases involved in the process of computational problem solving. 2.a. What is a user defined function? Give an example code. 2.b. Write a program to read the length and breadth of a rectangle. Find the perimeter and the area of the rectangle. 2.c. Consider a text file (t.txt) with the following contents. Write a program to display all the words that starts with a capital letter. PCPS final exam date is announced. We thought ESA paper will be easy. It was good. user defined func is defined by user. syntax is def funcname():

Archive page 3 from May__1SdbvafL_8.pdf; unit match candidate score 0.75; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 2.d. Describe briefly the characteristics of a string data structure. 3.a. Write a python code to demo this - Returning multiple elements in the same return statement from a function. 3.b. What is recursion? Which data structure is used while storing the recursive calls? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite Client code is as below: number = 7 print("The sum of first", number,"numbers is", find_sum(number)) 3.c. Find the output when executed. def f1(x,y,*arg,**kwarg): print(x+y) print(arg) print(kwarg) f1(2,3,56,33,34,4,6,a=21,b=31,c=61) immutable, slicing, indexing, concatenation, iterable recursion - function calling itself over and over until it solves the problem using base case . ||| uses stack 5 [56,33,34,4,6] {'a': 21, 'b': 31, 'c': 61}

Archive page 5 from May__1SdbvafL_8.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (5.0 Marks) 4.d. Define Generator in python. Explain with an example code to create generator objects in python. 5.a. Define Inheritance. Mention its types 5.b. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 Generator is an iterable object inheritance - child take stuff of parent 'is-a' & 'has-a' relation multi-level, single-level, multiple, hierarchical, hybrid

Archive page 1 from Dec__1enz7ur0jb.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 1/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - DEC 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Classify & group the following devices into Input/ Output/ Processing/ Storage/ Communication devices. Central Processing Unit, Modem, USB Stick, Microphone, Blue-Ray device, Speakers, Graphics Processing Unit, Touch screen, Projectors, Blue-tooth. 1.b. Evaluate the following expressions: i) a='rama'>'ravana'; print(a) ii) print("1" * 2, 2 * "1") iii) a = 20; b = 5; print(a&b) iv) 5(2 + 3 * 4) v) import math; print(math.sqrt(-25)) 1.c.

Write a Python program that prompts the user to enter positive integer values, and continues to prompt the user until a value of -1 is entered, adding up all the value entered and displaying the result.

Archive page 2 from Dec__1enz7ur0jb.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/8/24, 12:59 PM ESA - DEC 2023 -UE22CS151A about:blank 2/8 (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Explain the following: Algorithm Compiler Interpreter Semantic error Application software 2.a. Mention the properties of the list data structure. 2.b. Given s="python programming". What is the output of the following statements i) print (s[:]) ii) print (len(s)) iii) print (s[2:8]) iv) s[0]='T' print (s) v) k=s[0:6]+"not"+s[7:] print (k)

Archive page 1 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.8333; source text/OCR may be partial. (4.0 Marks) (7.0 Marks) (4.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JULY - 2023 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Explain the stages in the computational problem-solving process. 1.b. Answer the following: i) _____ is a built-in function that is used to determine the type of a Python object. ii) print(len(["ChatGPT", "Bard", 10, 2, 7.3, 20,77.0])). iii) print("My", "last", "Exam ", sep="*") iv) print(5 + (2 ** 5) // 10) v) The _____ operator is used for integer division. vi) Which function returns a random integer in range [a, b], including both end points in random module. (a and b are the start and end values for the range). vii) 7+5i. (State if the decimal literal is valid or invalid in python.) 1.c. Given the start and end numbers, write a python program to display all even numbers between the range. Sample Output: Enter the start of range: 6 Enter the end of range: 12 6 8 10 12

Archive page 2 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.d. A student's marks for five distinct subjects are taken using user input. Calculate and display the mark percentage. 2.a. Write a python program to read a string of 4 characters in lowercase from the user. Print every character of a string along with its uppercase letter and its ASCII/Unicode. Sample Output: Enter the string of 4 characters in lowercase: abcd a A 65 b B 66 c C 67 d D 68 2.b. Write a python program that takes the file name from the user and reads the contents(lines) of the file in the reverse order (from bottom). File1.txt contents: The laughter filled the room, infectious and joyous. It echoed and echoed, spreading happiness all around. It danced on the airwaves, filling hearts with warmth and light. Sample output: Enter file name: File1.txt It danced on the airwaves, filling hearts with warmth and light. It echoed and echoed, spreading happiness all around. The laughter filled the room, infectious and joyous.

Archive page 3 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (6.0 Marks) (5.0 Marks) (5.0 Marks) 2.c. Find the output of the following code snippets. i) t1 = (10,20,30,40,50) t2 = t1 + (60) t1 ii) t1=(100,200) t1*2 iii) d1=dict([(1,"Python"),(2,"Exam"),(3,"Over")]) print(d1,type(d1)) 2.d. Give any 5 built in functions of list in a tabular form with its description and code snippets 3.a. Write a function that accepts a string and calculate the number of upper-case letters and lower case letters in it.

Archive page 6 from July__1e0jrB6Yh8.pdf; unit match candidate score 0.5; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (3.0 Marks) 5.b. List the keywords in python with respect to Exception handling. 5.c. Find the output for the following python code. class Car: make = "TATA" def __init__(self,name,mileage): self.name = name self.mil = mileage

c1 = Car("Harrier",18) c2 = Car("Nexon",25) print(c1.name,c1.mil,c1.make) c2.name = "i20" c2.make = "Hyundai" print(c2.name,c2.mil,c2.make) 5.d. Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list (if any). Sample input: L=[10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80]

Archive page 1 from May__1rBeUIlse2.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (5.0 Marks) (6.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - May 2024 UE22CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of each statement when executed separately. i) a = 40; print("a") ii) print(c == 0) iii) a = 4; b = a; print(id(a) == id(b)) iv) print(2 * 3 * 2) v) print(False and True) 1.b. 1) Provide the binary equivalent of a given decimal number - 53 2) Define Translator. Name any two types of translators. 1.c. Write a program to print the following pattern. A B C D B C D C D D

Archive page 2 from May__1rBeUIlse2.pdf; unit match candidate score 0.5; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 1.d. Write a neat diagram to explain the phases involved in the process of computational problem solving. 2.a. What is a user defined function? Give an example code. 2.b. Write a program to read the length and breadth of a rectangle. Find the perimeter and the area of the rectangle. 2.c. Consider a text file (t.txt) with the following contents. Write a program to display all the words that starts with a capital letter. PCPS final exam date is announced. We thought ESA paper will be easy. It was good.

Archive page 3 from May__1rBeUIlse2.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (6.0 Marks) (5.0 Marks) 2.d. Describe briefly the characteristics of a string data structure. 3.a. Write a python code to demo this - Returning multiple elements in the same return statement from a function. 3.b. What is recursion? Which data structure is used while storing the recursive calls? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite Client code is as below: number = 7 print("The sum of first", number,"numbers is", find_sum(number)) 3.c. Find the output when executed. def f1(x,y,*arg,**kwarg): print(x+y) print(arg) print(kwarg) f1(2,3,56,33,34,4,6,a=21,b=31,c=61)

Archive page 5 from May__1rBeUIlse2.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) (5.0 Marks) 4.d. Define Generator in python. Explain with an example code to create generator objects in python. 5.a. Define Inheritance. Mention its types 5.b. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2

Archive page 1 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 1/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using the python 3.x i) print(print(0),end = "") ii) a = 3; b = a; print(a is b) iii) a = (3,4,5); print(type(a)) iv) a = 5; if a =10: print("It is ten") v) print(range(3,10,2)) 1.b. Say True or False. i) The function print can take any type of arguments. ii) The built in function, which returns the unique identity of the given object in python is ID() iii) Compiler executes one line of code at a time. iv) The function input() returns a string always. v) The syntax of a language is nothing but the set of rules to be followed in a language. 1.c. Take a whole number from the user. Display whether the entered number is perfect number or not. Expected output #1: Enter the number: 28 28 is a perfect number Expected output #2: Enter the number: 25 25 is not a perfect number Hint: A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. 1 0 None 2 True 3 tuple 4 syntax error 5 range(3, 10, 2) True Flase it's id() False it's interpreter True True

Archive page 2 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 2/7 (4.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keyword/s and variable/s from a given collection of identifiers. i) and ii) print iii) PriNt iv) input 1.e. With a neat diagram, explain the phases involved in the process of computational problem solving. 2.a. Mention any five characteristics of dictionary type in python. key: value, uses {key: value}, mutable, iterable, key should be immutable, sorted using hashing, doesnt support concatenation keyword and variable PriNt, print, input

Archive page 3 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.b. Perform the given operations separately on a sample list. li = [23, "yesterday", 44.6, 78.5, 900, "today", "tomorrow"] i) Create a new list with only strings from the given list. ii) Add a tuple t = (3,4,5) to the list li at the end to increment the length of the list by 1. Print the length of the list. 2.c. Develop a program to read the contents from one text file. Choose the words which are starting with a letter P and write only these words to another file line by line. Source file content is as below: Python is a Professional language which is the part of syllabus in PES university Output file content is as below: Python Professional PES 2.d. What happens when the below code is run? Modify the code to get the expected output. x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) Expected output: Inside 11 Outside 11 You get error as you can't directly change variables of outer scope. UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Archive page 4 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 4/7 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.e. Find the output of below mentioned code when executed using python 3.x. def my_function(p, q = 2, *arg, **kw): print(p); print(q); print(arg[1]) print(kw['b']); return arg print(my_function(2,3,10,14, a = 4,b = 5,c = 6)) 3.a.

Recursion is implemented using _____ to store the activation record of function calls in Last In First Out order. Fill the implementation of a recursive function to find the length of the string for a given client code. def length(s): #Fill this function suite #client code print(length("pesu-python")) 3.b. What is the type(m) in the below code? What gets printed when this code is run? def my_code1(): yield 6 yield 12 yield 18 m = my_code1() print(next(m), end = " ") print(next(m), end = " ") print(next(m)) print(next(m), end = " ") 2 3 14 5 (10,14) Stack class generator 6 12 18 error stopiteration

Archive page 6 from Jan - (Set-1)__1S8XvO2bSr.pdf; unit match candidate score 0.4167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 6/7 (5.0 Marks) (5.0 Marks) 4.a. Given list1 = [10,34,32,21,370,101]. Use list comprehension to do the following. i) Create a list of tuples where each tuple having an element from the given list and its square root. ii) Create a list of only even elements 4.b. #---------- class My_Exception(Exception): pass try: a = int(input()) b = int(input()) c = a/b if b == 5: raise My_Exception("Divisor cannot be 5") except ZeroDivisionError: print("Division by zero") except ValueError as v: print("Input must be an integer") except Exception: print("It is a problem if divisor is 5") except: print("error") else: print(c) i) If user enters 10 and 5 in two different lines, what gets printed? ii) If user enters 10 and 2 in two different lines, what gets printed? iii) If user enters 10 and five in two different lines, what gets printed? iv) If user enters 10 and 0 in two different lines, what gets printed? v) If user enters ten and press enters, what happens? It is a problem if divisor is 5 5 Division by zero Input must be an integer Input must be an integer

Archive page 1 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 1.0833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 1/8 (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using python 3.x i) print(print("choco",end = "")) ii) a = 3; b = 10; print(a and b) iii) a = {3,4,5}; print(type(a)) iv) a = 6; if a != 5: print("It is five") v) print(list(range(3,6))) 1.b. Say True or False. i) The word sep is used as keyword argument in input function. ii) The variable can have only digits to represent a value. iii) Interpreter executes one line of code at a time. iv) The membership operator returns a boolean value. v) a = 2 . This is an expression in python. chocoNone 10 set It is five [3, 4, 5] False False True True False Expression must give a result. like 2+3,etc ig. I am not sure for this question . because they tell even just 2 is an expression

Archive page 2 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.8333; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 2/8 (6.0 Marks) (4.0 Marks) (5.0 Marks) 1.c. Take a whole number from the user. Display whether the entered number is an Armstrong number or not. Expected output #1: Enter the number: 153 153 is an armstrong number Expected output #2: Enter the number: 154 154 is not an armstrong number Hint: Armstrong number is a positive number, if the sum of its own digits raised to the power of number of digits in a number gives the number itself. 1.d. Separate the keywords and variables from a given collection of identifiers. i) for ii) sum iii) Input iv)

def 1.e. List the two essences and three phases involved in the process of computational problem solving. keywords:for def variables:Input sum Representation, Algorithm Analysis, Design, Implementation, Testing

Archive page 3 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 3/8 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. Mention any five characteristics of set type in python 2.b. Perform the given operations separately on a sample dictionary d. d = {1: 111, 5: 555, 4:444, 7:777} i) Create a new list with only keys from the given dictionary. ii) Add a tuple t = (3,4,5) as key to the dictionary d with a value 345. iii)Pop any one item from the dictionary without specifying the key and print the popped item. 2.c. Develop a program to read only first line from a text file. Choose the words which are starting with a capital letter and write only these words to another file line by line. Source file content is as below: Python is a Professional course in PES university It is one of the Popular Language. Output file: Python Professional PES mutable, unordered, no duplicates iterable, uses {a,b,..} pop - needs key popitem - pops random and gives output as tuple (key, value)

Archive page 4 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 4/8 (4.0 Marks) (5.0 Marks) (5.0 Marks) 2.d. What happens when the below code is run? Modify the code to get the expected output. f1(2) f1(33) f1(12,[3,4]) f1(25) def f1(a,b=[]): b.append(a) print(b) Expected output: [2] [2, 33] [3, 4, 12] [2, 33, 25] 2.e. Find the output when the below code is executed using python 3.x. def my_function(p, q, *arg, **kw): print(p); print(q); print(arg[0]); print(kw['a']); return list(kw.values()) print(my_function(2,3,10,14, a = 400,b = 500,c = 600)) 3.a. What is recursion? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite #Client code number = 10 print("The sum of first", number,"numbers is", find_sum(number)) error as f1 is not defined when the function call is done. first should def f1 then should be func calling 2 3 10 400 [400,500,600] Repeat

Archive page 8 from Jan - (Set-2)__1HZjDaXNmS.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 8/8 (4.0 Marks) (5.0 Marks) 4.d. List any four types of is – a relationship supported by Python. 4.e. Fill the implementation of a new type – Chocolate to get the output as expected for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2 single-level, multi-level, heirarchial, hybrid, multiple repeat

Archive page 1 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 1.0; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 1/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using the python 3.x

i) print(print(0),end = "") ii) a = 3; b = a; print(a is b) iii) a = (3,4,5); print(type(a)) iv) a = 5; if a =10: print("It is ten") v) print(range(3,10,2)) 1.b. Say True or False. i) The function print can take any type of arguments. ii) The built in function, which returns the unique identity of the given object in python is ID() iii) Compiler executes one line of code at a time. iv) The function input() returns a string always. v) The syntax of a language is nothing but the set of rules to be followed in a language. 1.c. Take a whole number from the user. Display whether the entered number is perfect number or not. Expected output #1: Enter the number: 28 28 is a perfect number Expected output #2: Enter the number: 25 25 is not a perfect number Hint: A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself.

Archive page 2 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 2/7 (4.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keyword/s and variable/s from a given collection of identifiers. i) and ii) print iii) PriNt iv) input 1.e. With a neat diagram, explain the phases involved in the process of computational problem solving. 2.a. Mention any five characteristics of dictionary type in python.

Archive page 3 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.b. Perform the given operations separately on a sample list. li = [23, "yesterday", 44.6, 78.5, 900, "today", "tomorrow"] i) Create a new list with only strings from the given list. ii) Add a tuple t = (3,4,5) to the list li at the end to increment the length of the list by 1. Print the length of the list. 2.c. Develop a program to read the contents from one text file. Choose the words which are starting with a letter P and write only these words to another file line by line. Source file content is as below: Python is a Professional language which is the part of syllabus in PES university Output file content is as below: Python Professional PES 2.d. What happens when the below code is run? Modify the code to get the expected output. x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) Expected output: Inside 11 Outside 11

Archive page 4 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 4/7 (5.0 Marks) (5.0 Marks) (5.0 Marks) 2.e. Find the output of below mentioned code when executed using python 3.x. def my_function(p, q = 2, *arg, **kw): print(p); print(q); print(arg[1]) print(kw['b']); return arg print(my_function(2,3,10,14, a = 4,b = 5,c = 6)) 3.a. Recursion is implemented using _____ to store the activation record of function calls in Last In First Out order. Fill the implementation of a recursive function to find the length of the string for a given client code. def length(s): #Fill this function suite #client code print(length("pesu-python")) 3.b. What is the type(m) in the below code? What gets printed when this code is run? def my_code1(): yield 6 yield 12 yield 18 m = my_code1() print(next(m), end = " ") print(next(m), end = " ") print(next(m)) print(next(m), end = " ")

Archive page 6 from Jan - (Set-1)__1kzIfeDFc9.pdf; unit match candidate score 0.4167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 1) about:blank 6/7 (5.0 Marks) (5.0 Marks) 4.a. Given list1 = [10,34,32,21,370,101]. Use list comprehension to do the following. i) Create a list of tuples where each tuple having an element

from the given list and its square root. ii) Create a list of only even elements 4.b. #---------- class My_Exception(Exception): pass try: a = int(input()) b = int(input()) c = a/b if b == 5: raise My_Exception("Divisor cannot be 5") except ZeroDivisionError: print("Division by zero") except ValueError as v: print("Input must be an integer") except Exception: print("It is a problem if divisor is 5") except: print("error") else: print(c) i) If user enters 10 and 5 in two different lines, what gets printed? ii) If user enters 10 and 2 in two different lines, what gets printed? iii) If user enters 10 and five in two different lines, what gets printed? iv) If user enters 10 and 0 in two different lines, what gets printed? v) If user enters ten and press enters, what happens?

Archive page 1 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 1.0833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 1/8 (5.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Print the output of the following expressions when executed each statement separately using python 3.x i) print(print("choco",end = "")) ii) a = 3; b = 10; print(a and b) iii) a = {3,4,5}; print(type(a)) iv) a = 6; if a != 5: print("It is five") v) print(list(range(3,6))) 1.b. Say True or False. i) The word sep is used as keyword argument in input function. ii) The variable can have only digits to represent a value. iii) Interpreter executes one line of code at a time. iv) The membership operator returns a boolean value. v) a = 2 . This is an expression in python.

Archive page 2 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 2/8 (6.0 Marks) (4.0 Marks) (5.0 Marks) 1.c. Take a whole number from the user. Display whether the entered number is an Armstrong number or not. Expected output #1: Enter the number: 153 153 is an armstrong number Expected output #2: Enter the number: 154 154 is not an armstrong number Hint: Armstrong number is a positive number, if the sum of its own digits raised to the power of number of digits in a number gives the number itself. 1.d. Separate the keywords and variables from a given collection of identifiers. i) for ii) sum iii) Input iv) def 1.e. List the two essences and three phases involved in the process of computational problem solving.

Archive page 3 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 3/8 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. Mention any five characteristics of set type in python 2.b. Perform the given operations separately on a sample dictionary d. d = {1: 111, 5: 555, 4:444, 7:777} i) Create a new list with only keys from the given dictionary. ii) Add a tuple t = (3,4,5) as key to the dictionary d with a value 345. iii)Pop any one item from the dictionary without specifying the key and print the popped item. 2.c. Develop a program to read only first line from a text file. Choose the words which are starting with a capital letter and write only these words to another file line by line. Source file content is as below: Python is a Professional course in PES university It is one of the Popular Language. Output file: Python Professional PES

Archive page 4 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 4/8 (4.0 Marks) (5.0 Marks) (5.0 Marks) 2.d. What happens when the below code is

run? Modify the code to get the expected output. f1(2) f1(33) f1(12,[3,4]) f1(25) def f1(a,b=[]): b.append(a) print(b) Expected output: [2] [2, 33] [3, 4, 12] [2, 33, 25] 2.e. Find the output when the below code is executed using python 3.x. def my_function(p, q, *arg, **kw): print(p); print(q); print(arg[0]); print(kw['a']); return list(kw.values()) print(my_function(2,3,10,14, a = 400,b = 500,c = 600)) 3.a. What is recursion? Fill the implementation of a recursive function to find the sum of first n natural numbers for a given client code. def find_sum(n): #Fill this function suite #Client code number = 10 print("The sum of first", number,"numbers is", find_sum(number))

Archive page 8 from Jan - (Set-2)__1YxZdNR8Ws.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:32 AM ESA -JAN 2024 - UE23CS151A (set- 2) about:blank 8/8 (4.0 Marks) (5.0 Marks) 4.d. List any four types of is – a relationship supported by Python. 4.e. Fill the implementation of a new type – Chocolate to get the output as expected for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2

Archive page 1 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 1/7 (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. i) How do compilers and interpreters differ in their approach to translating code? (2 marks) ii) List any 3 input devices. (3 marks) 1.b. Answer the following. i) _____ is a built-in function that is used to determine the type of a python object. ii) print("My", "last", "Exam ", sep="*") iii) The _____ operator is used for integer division. iv) print(15 & 3) v) 7+5i. (State if the literal is valid or invalid in python.) vi) print(True == 1)

Archive page 2 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.9167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 2/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.c. Given the below python code, x = 10 while x > 5: x -= 2 print(x) i) Find the output of the code – 1 mark ii) If you change the initial value of "x" to 6, what will be the output? -- 2 marks iii) What happens if “x -= 2” line is removed from the code. -- 2 marks 1.d. i) Find the output - 2 marks name = "PESU" age = 50 is_active = True if name == "PESIT" or (name == "PESU" and age <= 30 and is_active): print("Welcome") else: print("Access denied") ii) Brief the concept of short-circuit evaluation wrt Python's Logical operators. - 3 marks 1.e. Write a program that takes the number of rows 'n' as input from the user and prints a diamond pattern using asterisks (*) as shown in the sample output. Note that input ‘n’ will be an odd integer. The pattern should have a diamond shape with 'n' rows in the middle. If n is 5, the expected pattern is as below. * *** ***** *** *

Archive page 3 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 3/7 (4.0 Marks) (5.0 Marks) (6.0 Marks) 2.a. List any four characteristics of a set. 2.b.

Given the multiline string student_scores = """pes001 98 78 67 pes002 99 44 79 pes006 44 55 33 pes007 55 88 40 pes009 34 78 65""" Each row has details of a student (srn, phy_marks, chem_marks, math_marks) separated by a space n between. Calculate the total marks of a particular student. Display the srn as the key and the total marks as the value. Expected Output: {'pes001': 243, 'pes002': 222, 'pes006': 132, 'pes007': 183, 'pes009': 177} 2.c. Write a python program that takes a sentence as input and converts each alphabet in a given sentence to the next letter in the alphabet, while preserving the case of the letters. For example, a is converted to b, b to c, so on and z to a. Expected Output: Enter a sentence: hello how are you Converted sentence: ifmmp ipx bsf zpv

Archive page 4 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5833; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 4/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.d. Write a python program to open a file, iterate through its contents line by line, print each line along with its corresponding line number, and then close the file. 2.e. Find the output def example(a, b=10, *args, **kwargs): print(a,b) print(args) print(kwargs) print(type(args),type(kwargs)) example(1, 2, 3, "Python", name="PESU", city="Blr") 3.a. Define Recursion. Write a recursive function to find the gcd of two whole numbers. These two whole numbers are sent from the client code to this function.

Archive page 5 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 5/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 3.b. Given a list1 = ['23','909','44','23','66','720', '9'] Perform sorting of these elements in a list using ascending order based on the last digit of every element. Expected output: ['720', '23', '23', '44', '66', '909', '9'] 3.c. i) Mention any two characteristics of closures. (2 marks) ii) Create a generator function that yields cubes of numbers from 1 to n. Accept n from the user. Test this generator function. (3 marks) 3.d. i)To install a third-party module in Python, the preferred command is _____. ii) _____ is the built in global variable which contains the document string of the function specified or the module itself. iii) _____ is the module which allows writing test cases as a part of document sting and runs those test cases using the function in that module iv) testadd is a valid method name which pytest can recognize. State True or False

Archive page 7 from Jan - (Set-3)__1ghwvCbUda.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 3) about:blank 7/7 (6.0 Marks) (2.0 Marks) (5.0 Marks) 4.c. i) Explain constructors with an example code. (2 marks) ii) Create a super class named X and a subclass named Y. Y class inherits the properties of the X class. Create a constructor to initialize two lists in X. Write a method in the subclass that concatenates both the lists and displays the concatenated list. (4 marks) Sample input: list1=[10,20,30] list2=[40,50,60] Expected output: [10, 20, 30, 40, 50, 60] 4.d. Find the output list1 = [10,20,30] list2 = ['a', 'b', 'c', 'd'] list3 = [100,200] result = list(zip(list1, list2, list3)) print(result) 4.e. What is an exception? Mention it's categories. List any four built-in Exceptions.

Archive page 1 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.75; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 1/7 (5.0 Marks) (6.0 Marks) PES University, Bengaluru (Established under Karnataka

Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - JAN 2023 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. i) How do the functions of main memory and secondary memory differ in a computer system ? (2 marks) ii) List any 3 output devices. (3 marks) 1.b. Answer the following: i) What is the output of the expression type(3.5+2) ii) print("Pesu", "is", "the", "Best ", sep="#") iii) The _____ operator is used for shifting bits right. iv) print(15 | 0) v) 4+6j. (State if the decimal literal is valid or invalid in python.) vi) print(False == 0)

Archive page 2 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 2/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 1.c. Given the below python code i = 1 while i <= 5: print(i * i) i += 2 i) Find the output of the code – 2 marks ii) If you change the initial value of "i" to 4 in the beginning, what will be the output? -- 2 marks iii)What happens if “i += 2” line is removed from the code. -- 1 mark 1.d. i) How does Python handle nested if statements? Provide an example. (3 marks) ii) Find the output of the below code. (2 marks) x = 5 y = 10 z = 15 if x < y and y < z: print("Good Luck") else: print("Best Wishes") if x == y and y != z: print("All the best") else: print("Best of Luck") 1.e. Write a program that takes the number of rows ‘n’ as input and prints the following pattern. For n=4, Output is as below. 1 10 101 1010

Archive page 3 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 3/7 (5.0 Marks) (6.0 Marks) (4.0 Marks) 2.a. Given the multiline string student_scores= """pes001 98 78 67 pes002 99 44 79 pes001 44 55 33 pes003 55 88 40 pes002 33 78 51 pes005 34 28 65 pes006 88 61 53""" Each row has details of a student (srn, phy_marks, chem_marks, math_marks) separated by a space in between. Remove the duplicate srn’s in the string, display the unique srn and the count of it. Expected Output: {'pes007', 'pes001', 'pes009', 'pes006', 'pes002'} The count is: 5 2.b. Write a python function that accepts two strings and tests if two strings are anagrams of each other. Note: An anagram is a word or phrase formed by rearranging the letters of another word Sample output: 1 Enter the first word: ink Enter the second word: kin True Sample output: 2 Enter the first word: pac Enter the second word: rac False 2.c. List any four characteristics of a list in python.

Archive page 4 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 4/7 (5.0 Marks) (5.0 Marks) (6.0 Marks) 2.d. Write a python program that takes the file name from the user and reads all the contents of the file. Display these contents on the terminal from the last line of the file to the first line. 2.e. Find the output def test(a, b=10, *args, **kwargs): print(a + b) print(type(b),type(args)) print(args) print(kwargs) test("Good","Day", 3, "Python",5+4j, city="Blr", state="KA") 3.a. Mention the two criteria of recursion. Write a recursive function to find the the length of the string and test this function.

Archive page 5 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.5; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 5/7 (5.0 Marks) (5.0 Marks) (4.0 Marks) 3.b. The list has a mixture of integers and strings as shown here. list1 = [23,909,'54','19','66','720', '93'] Create a new sorted list in ascending order based on the second digit of every element in the list. Expected output: [909, '720', 23, '93',

'54', '66', '19'] 3.c. i) Define Generators. (2 marks) ii) Write a program to generate odd numbers using closure. (3 marks) 3.d. i)The keyword used to include an entire module into a python code is _____. ii) _____ is the built in global variable which contains the name of the module. iii)testone.py is a valid pytest filename. State True or False. iv)_____ means a successful test case when you run the file which is recognized by the pytest module

Archive page 6 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.9167; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 6/7 (5.0 Marks) (6.0 Marks) (6.0 Marks) 3.e. Define the following along with their syntax in the tkinter module. i) Entry() (2 marks) ii) Label() (2 marks) iii) mainloop() (1 mark) 4.a. A. Given the list, perform the operations using functional programming constructs list1 = ['Java', 'Python', 'Csharp', 'Javascript', 'Go', 'Swift'] i) Convert the strings in a list to lowercase if the length of it less than 4. Then display it. (2 marks) ii) Display all the strings in the list which has the letter o in it. (2 marks) B. What is the purpose of reduce function from Python's functools module? (2 marks) 4.b. i) Find the output: (2 marks) numbers = [1, 2, 3, 4, 5] L = [x**2 for x in numbers if x % 2 != 0] print(L, type(L)) ii) Find the output: (2 marks) words = ["tiger", "leopard", "elephant", "lion"] count = [sum(1 for letter in word if letter not in 'aeiou') for word in words] print(count) iii) Write a function using lambda that returns the reverse of a string passed to it. (2 marks)

Archive page 7 from Jan - (Set-4)__1cbShhBdIG.pdf; unit match candidate score 0.6667; source text/OCR may be partial. 11/11/24, 11:33 AM ESA -JAN 2024 - UE23CS151A (set- 4) about:blank 7/7 (6.0 Marks) (2.0 Marks) (5.0 Marks) 4.c. i) Brief about destructors with an example code. (2 marks) ii)Create a super class named A and a subclass named B. B class inherits the properties of the A class. The super class has an attribute called list L, which is homogeneous and contains integers. List may contain duplicate elements. Write a program such that the subclass removes the duplicate elements from the list nu adding a function the subclass(if any). (4 marks) Sample input: L=[10,20,30,30,20,90,10,80] Expected output: L = [10,20,30,90,80] 4.d. Find the output list1 = [1, 2, 3, 4] list2 = ['a', 'b', 'c', 'd'] list3 = ['x', 'y'] result = list(zip(list1, list2, list3)) print(result) 4.e. List the keywords of python used in Exception handling.

Archive page 1 from June__1ba82WHmaJ.pdf; unit match candidate score 0.9167; source text/OCR may be partial. (6.0 Marks) (4.0 Marks) (5.0 Marks) PES University, Bengaluru (Established under Karnataka Act 16 of 2013) END SEMESTER ASSESSMENT (ESA) - Jan - June 2024 UE23CS151A - Python for Computational Problem Solving Total Marks : 100.0 1.a. Mention the output of the following expressions when executed each statement separately using the python 3.x. i) print((1),end = "") ii) a = 3; b = 0; print(a is not b) iii) a = {3,4,5}; print(type(a)) iv) a = 5; if a == 10: print("It is ten") else: print("It is not ten") v) print(list(range(2,15,3))) vi) print(p = 2) 1.b. Say True or False.

i) The print() function in python can take any number of arguments. ii) Python supports Object Oriented Programming paradigm. iii) Compiler checks for syntactical errors in the code. iv) The function print() returns a string always. 1.c. Take a whole number from the user. Display

appropriate message by checking whether the entered number is a multiple of 10 or not. Expected output #1: Enter the number: 28 28 is not a multiple of 10 Expected output #2: Enter the number: 250 28 is a multiple of 10 Hint: A whole number is a positive integer starting from 0.

Archive page 2 from June__1ba82WHmaJ.pdf; unit match candidate score 0.6667; source text/OCR may be partial. (5.0 Marks) (5.0 Marks) (5.0 Marks) (5.0 Marks) 1.d. Separate the keywords and variables from a given collection of identifiers. i) or ii) If iii) choco iv) pass v) break 1.e. List any three phases involved in the process of computational problem solving. Explain any one in detail. 2.a. Mention any five characteristics of list type in python. 2.b. Perform the given operations separately on a sample list. Sample_li = ["23", "yesterday", "44.6", "78.5", "900", "today", "tomorrow"] i) Create a new list with only first letter from the given list of strings and print the new list. #3 MARKS ii) Append the elements of the tuple t = (3,4,5) to the Sample_li using a single statement and print the Sample_li. #2 MARKS

Archive page 3 from June__1ba82WHmaJ.pdf; unit match candidate score 0.4167; source text/OCR may be partial. (6.0 Marks) (4.0 Marks) (5.0 Marks) 2.c. Write a program to read all the contents from one text file and count the number of times the word ‘sea’ is repeated? Print the count to a new file. Source file content is as below: A sailor went to sea, sea, sea To see what he could see, see, see But all that he could see, see, see Was the bottom of the deep blue sea, sea, sea! Output file: 6 2.d. Find the output of below code. i) x=10 def f1(): x=x+1 print("Inside",x) f1() print("Outside",x) ii) x=10 def f1(): global x x=x+1 print("Inside",x) f1() print("Outside",x) 2.e. Find the output when the below code is executed using python 3.x. def my_func(p, q = 20, **kw): print(p); print(q); print(kw['a']); return type(kw) print(my_func(2, a = 4,b = 5,c = 6))

Archive page 6 from June__1ba82WHmaJ.pdf; unit match candidate score 0.5833; source text/OCR may be partial. (4.0 Marks) (5.0 Marks) 4.d. List the features of Object oriented Programming languages. 4.e. Fill the implementation of a new type – Chocolate to get the output as shown for the given client code. class Chocolate: #Fill this class #client code c1 = Chocolate("sweet", "imli") print("Number of chocolates:", Chocolate.count) c2 = Chocolate("sugary", "salty") c3 = Chocolate("imly", "sweety") print("Number of chocolates:", Chocolate.count) print(c2) del c1 print("Number of chocolates:", Chocolate.count) #expected output Number of chocolates: 1 Number of chocolates: 3 sugary salty Number of chocolates: 2

EV0001230 · EV0001231 · EV0001232 · +55 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Introduction transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Introduction transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Introduction__ca07fd2e558c8a7c.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Man Cabbage Goat Wolf Problem transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9988; only the JSON segmentation supplies the displayed timestamps.
Man Cabbage Goat Wolf Problem transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/001_Man_Cabbage_Goat_Wolf_Problem__1ef4a632cd03282d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9971; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__2849d807580cb7ae.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
Process of Computation Problem Solving - Analysis, Design, Implementation, Testing transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/003_Process_of_Computation_Problem_Solving_-_Analysis_Design_Implementation_Testing__c5451c431739faf6.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Computer Hardware and software transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
Computer Hardware and software transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_Hardware_and_software__d0cc7ac4df04ef9f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Computer hardware transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Computer hardware transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/004_Computer_hardware__845ddeaeeec94ce6.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9932; only the JSON segmentation supplies the displayed timestamps.
Program Structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_Structure__c33bde9f45f13d1b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Program structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__141e953573b7fab3.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Program structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/008_Program_structure__c191b948fb0cbd98.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
print function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
print function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
print function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
print function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/009_print_function__fa29381f8beff681.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Variable video 1 transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9991; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Variable video 1 transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_Variable_video_1__6a17ee49c6f79e16.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9986; only the JSON segmentation supplies the displayed timestamps.
variable and id function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/010_variable_and_id_function__92e3df0e8b11255d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
input function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
input function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__729751eddd0f266f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
input function transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
input function transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/011_input_function__c33256eff090b5ec.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Arithmetic, membership and relational operators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9992; only the JSON segmentation supplies the displayed timestamps.
Arithmetic, membership and relational operators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Arithmetic_membership_and_relational_operators__ff38db0dde601b27.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Bitwise operators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9996; only the JSON segmentation supplies the displayed timestamps.
Bitwise operators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_Bitwise_operators__45d6586e889881f1.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9969; only the JSON segmentation supplies the displayed timestamps.
logical operators and short circuit evaluation transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_logical_operators_and_short_circuit_evaluation__7ee47e67d3dc874f.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
operators and expressions transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
operators and expressions transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/012_operators_and_expressions__c32527d44167ebb9.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
sequential, selection control transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
sequential, selection control transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/015_sequential_selection_control__3c385369aff92ed0.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
iterative control structure transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/016_iterative_control_structure__f5f1a1d31c8dff99.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
String Operations transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9989; only the JSON segmentation supplies the displayed timestamps.
String Operations transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/AV_Summaries/041_String_Operations__8206ac9c8ddf207b.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE23CS151A PCPS Unit 3 Sowmya Shree Sindhu Class 83 84 85 Modules (converted PDF; original hash d6dc37db35a480942e7cde93c54bf6cad030db9b925d2ee71ebc02d5091c4f25)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE23CS151A_PCPS_Unit_3_Sowmya_Shree_Sindhu_Class__83_84_85_Modules__5aae83d0a0551803.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 1 2 Intro to title (converted PDF; original hash 7eaea6a237d5be0a996eadc48813107306d0efdca313cf9fd092a18231688315)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__1__2_Intro_to_title__37baecfa2540966d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 14 15 Modules (converted PDF; original hash 5f23fb8257ff88f8822fddae23f377b282ffb7c2b945a66003f796669d7078e3)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__14__15_Modules__85ccca0fe5d7d2c4.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 3 Process of CPS (converted PDF; original hash 5bb4abd3e907609962af7a67d252efb563a03696068503cf2b4a2344063789fbb)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/003_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__3_Process_of_CPS__3f3254779c6a452a.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 4 5 Computer Hardware software (converted PDF; original hash 19e83e41e1f18720140876bf601405c14698c9efe95c3f4f43d1f9d0316ea332)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/004_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__4_5_Computer_Hardware_software__57579a8e9ad25cbb.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 6 Intro to Programming Languages (converted PDF; original hash 4818457e456ef5ddbd6e665e8631502f553418ef564bf0718f5f50501ef3f818)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/006_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__6_Intro_to_Programming_Languages__0cbeda79209c1dbf.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 7 Syntax semantics translation models (converted PDF; original hash bd5b4881571bd02e47afa988e5e24e089bbf620bc9bd823b27c88a762252fb02)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/007_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__7_Syntax_semantics_translation_models__656e0afe2d8d3282.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Installation and execution (converted PDF; original hash ae7555a2800a32932355a1e1b7e7fcc203aac277b798052079a43edb274bc3c8)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Installation_and_execution__2fbbee57dfe57679.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 8 Program structure (converted PDF; original hash c4f94ab07e4c657e4877f4613ecf7a2fe96eed53a554092a38d82844b5c3cb73)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/008_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__8_Program_structure__ed4abb0db838da93.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: IDLE	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: IDLE	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for IDLE (converted PDF; original hash 7dc0bdb848c62ce25230b2a2552529bce68cd6ed9140c11fa060db5ad16d9e90)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_IDLE__b0a7da958de0835d.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 9 output function print (converted PDF; original hash aa9c058d97b323eb3f1bce810006ad283b52dad346b0bdde16d9c5b7fbea7bb2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/009_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__9_output_function_print__1ca49294268909b9.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 10 variable id type (converted PDF; original hash a2054b399002f95eb5351ec8f47676d24c7180c4f0618cae39011724cc9ea66f)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/010_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__10_variable_id_type__b024fe65bb8bef33.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 11 input function (converted PDF; original hash 789fa22f6e8813727f06aa92ca42305fa9663a997a14017bf8a1291cddd60160)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/011_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__11_input_function__fe2e37a1fa374eaf.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 12 Operators Part1 (converted PDF; original hash 9c50d1cabe8737f8b6ce34cd242f300e169cdffbee523d31ac671b45fe50b1f2)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/012_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__12_Operators_Part1__b53129837a8bbfbd.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Operator Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Operator Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for Operator Polymorphism (converted PDF; original hash f73f31e80dd965d432bfa3e587c4f6296d49489a6ee16b86f6c64eb959b0faf4)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_Polymorphism__32ad541949df1cab.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Operator precedence and associativity	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Operator precedence and associativity	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Operator precedence and associativity (converted PDF; original hash 25fedd35c1a45740981f8e60315c4ba28620516f88089827b68da2057257ea75)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/014_Operator_precedence_and_associativity__d91f176414ae90ad.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 1 SindhuRPai Class 15 16 Control structures (converted PDF; original hash 9f9dc7b6245dd3f58cf0c0e4592bf57ecda60736fb392763d5a1e8495f4bfd44)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/015_UE24CS151A_PCPS_Unit_1_SindhuRPai_Class__15_16_Control_structures__bcb5c5877df41eba.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 1 General intro to data structures in python (converted PDF; original hash f9c2774ed4c06b020b330361d708d4c9dbddf28f7eb5e28c3e2e2687130ef91d)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/027_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__1_General_intro_to_data_structures_in_python__a531eb93c7c504ed.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 9 10 Strings types operations (converted PDF; original hash c38d2f28c14f3eae41c162f2a5c12da5e0ae9dd217dd3bc58f9bbb806f156513)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/041_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__9_10_Strings_types_operations__31d67ccee1eda8aa.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 2 SindhuRPai Class 2 lists (converted PDF; original hash db25dac784ce6be4ce8de74c6ed9167a461035fdd661176317408e6595a1ed55)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_01/Slides/044_UE24CS151A_PCPS_Unit_2_SindhuRPai_Class__2_lists__05be1d07364e2634.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Extracted historical pages: Dec__1aef5kbQ-b.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/Dec__1aef5kbQ-b.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Source	Path	Use
Extracted historical pages: July__17cJZWr8dZ.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/July__17cJZWr8dZ.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: May__1SdbvafL_8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Answers/May__1SdbvafL_8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Dec__1enz7ur0jb.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/Dec__1enz7ur0jb.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: July__1e0jrB6Yh8.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/July__1e0jrB6Yh8.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: May__1rBeUIlse2.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE22CS151A/May__1rBeUIlse2.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-1)__1S8XvO2bSr.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Answers/Jan - (Set-1)__1S8XvO2bSr.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-2)__1HZjDaXNmS.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Answers/Jan - (Set-2)__1HZjDaXNmS.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-1)__1kzIfeDFc9.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-1)__1kzIfeDFc9.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-2)__1YxZdNR8Ws.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-2)__1YxZdNR8Ws.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Source	Path	Use
Extracted historical pages: Jan - (Set-3)__1ghwvCbUda.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-3)__1ghwvCbUda.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: Jan - (Set-4)__1cbShhBdIG.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/Jan - (Set-4)__1cbShhBdIG.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.
Extracted historical pages: June__1ba82WHmaJ.pdf	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/First_Year_8enterprise/Files/Python/Papers/ESA Papers/UE23CS151A/June__1ba82WHmaJ.pdf	Candidate-matched page text excerpt; original PDF bytes, hash and path remain in datasets/question-archive.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 2 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 16 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00001	1 covered · 0 excluded · 0 gaps	catalogue: Unit title and index copied from the official units.json catalogue; human content review is pending.

Source	Unit counts	Example decisions
SRC00002	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare...
SRC00003	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare...
SRC00004	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00005	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0035. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0036 to s0067. Sidecar variants compare...
SRC00006	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00007	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare...
SRC00008	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00009	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0047. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0048 to s0066. Sidecar variants compare...
SRC00010	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00011	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare...
SRC00012	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00013	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0026. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0027 to s0050. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0051 to s0056. Sidecar variants compare...
SRC00014	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00015	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare...
SRC00016	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00017	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0046. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0047 to s0063. Sidecar variants compare...
SRC00018	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00019	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare...
SRC00020	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00021	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0057. Sidecar variants compare...
SRC00022	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00023	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare...
SRC00024	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00025	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0054. Sidecar variants compare...
SRC00026	1 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare...
SRC00027	1 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare...
SRC00028	1 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare...
SRC00029	1 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0005. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00030	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...
SRC00031	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00032	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...
SRC00033	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00034	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00035	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00036	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...
SRC00037	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0061. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00038	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare...
SRC00039	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00040	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare...
SRC00041	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0023. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0024 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0066. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00042	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare...
SRC00043	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00044	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare...
SRC00045	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0044. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00046	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare...
SRC00047	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare...
SRC00048	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00049	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare...
SRC00050	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00051	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare...
SRC00052	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00053	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0060. Sidecar variants compare...
SRC00054	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00055	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00056	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00057	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00058	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00059	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...
SRC00060	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00061	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0058. Sidecar variants compare...
SRC00062	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00063	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare...
SRC00064	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00065	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0039. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0040 to s0059. Sidecar variants compare...
SRC00066	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare...
SRC00067	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00068	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare...
SRC00069	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0030. Sidecar variants compare...
SRC00070	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00071	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare...
SRC00072	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00073	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0043. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0044 to s0063. Sidecar variants compare...
SRC00074	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00075	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare...
SRC00076	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00077	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0056. Sidecar variants compare...
SRC00078	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00079	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare...
SRC00080	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00081	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0018. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0019 to s0038. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0039 to s0054. Sidecar variants compare...
SRC00082	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00083	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00084	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00085	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00086	30 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00088	30 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00087	30 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00089	18 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00091	18 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00090	18 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00092	27 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00094	27 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00093	27 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00095	12 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00097	12 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00096	12 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00098	13 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00100	13 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00099	13 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00101	10 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00103	10 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00102	10 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00104	8 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00106	8 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00105	8 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00107	4 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00109	4 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00108	4 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00110	6 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00112	6 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00111	6 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00113	27 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00115	27 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00114	27 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00116	7 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00118	7 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00117	7 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00119	12 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00121	12 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00120	12 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00122	5 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00124	5 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00123	5 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00125	23 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00127	23 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00126	23 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00128	6 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00130	6 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00129	6 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00131	7 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00133	7 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00132	7 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00134	20 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00136	20 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00135	20 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00137	7 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00139	7 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00138	7 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00140	30 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00142	30 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00141	30 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00143	28 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00145	28 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00144	28 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00146	3 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00147	4 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00148	4 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.

Source	Unit counts	Example decisions
SRC00149	2 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed.
SRC00150	4 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00151	4 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.

Source	Unit counts	Example decisions
SRC00152	5 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00153	5 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00154	5 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.

Source	Unit counts	Example decisions
SRC00155	5 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00156	6 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.
SRC00157	7 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.

Source	Unit counts	Example decisions
SRC00158	4 covered · 0 excluded · 0 gaps	q1: Original archive PDF page 1; candidate unit association is text-based and unconfirmed. q2: Original archive PDF page 2; candidate unit association is text-based and unconfirmed. q3: Original archive PDF page 3; candidate unit association is text-based and unconfirmed.

C Revision and retrieval practice

Rapid recall

What is the difference between precedence and associativity? Precedence ranks different operators; associativity chooses grouping among operators at the same precedence. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json

Why does a script use print() for a result? A script does not echo a bare expression the way the interactive interpreter does; print() writes the value to the terminal. EV0000246 · EV0000247 · EV0000248 · +37 source units in pack.json